

SECTION 1 — HELM

What It Is

Helm is a package manager for Kubernetes — like `yum` for RedHat or `apt` for Ubuntu, but for K8s.

Why It Exists

Installing apps on Kubernetes needs many YAML files. Helm bundles them into one package called a Chart.

Think of It Like This

Helm is to Kubernetes what an App Store is to your phone — one command installs everything.

Key Points

- Chart = a collection of manifest files in a folder structure
- Release = a running instance of a chart with specific config
- Repository = online store where charts are published
- Helm is written in Go and talks to Kubernetes via the API server

Architecture

1. HELM REPOSITORY	→ Has all public Helm charts
2. HELM CLIENT	→ Downloads charts from repo
3. API SERVER	→ Executes charts on cluster

Commands — Install Helm

```
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
```

Interview Answer

"In practice, Helm saves enormous time because instead of applying 10 separate YAML files, I run one install command and everything deploys

together. The reason it matters is consistency — the same chart deploys identically across dev, staging, and production environments."

Faculty Notes Explanation:

HELM:

In K8S Helm is a package manager to install packages
in Redhat: yum & Ubuntu: apt & K8s: helm

it is used to install applications on clusters.
we can install and deploy applications by using helm
it manages k8s resources packages through charts
chart is a collection of files organized on a directory
structure.
chart is collection of manifest files.
a running instance of a chart with a specific config is called
a release.
The Helm client and library is written in the Go programming
language.
The library uses the Kubernetes client library to communicate
with Kubernetes.

ARCHITECTURE:

1. HELM REPOSITORY: IT HAS ALL HELM REPOS WHICH IS PUBLICALLY AVAILABLE
2. HELM CLIENT: DOWNLOADS HELM CHARTS FROM HELM REPOS.
3. API SERVER: DOWNLOADED HELM CHARTS WILL BE EXECUTED ON CLUSTER WITH API SERVER.

Install HELM

```
-----  
curl -fsSL -o get_helm.sh  
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3  
chmod 700 get_helm.sh  
./get_helm.sh  
helm version
```

SECTION 2 — ArgoCD

What It Is

ArgoCD is a GitOps continuous delivery tool that automatically deploys your Kubernetes apps from a Git repository.

Why It Exists

CI/CD pipelines are not ideal for Kubernetes deployments. ArgoCD watches your Git repo and syncs changes to the cluster automatically.

Think of It Like This

ArgoCD is like a security guard who watches your Git repo 24/7 — the moment something changes, it updates the cluster immediately.

Key Points

- GitOps tool — Git is the single source of truth
- Monitors Git repo for changes → applies them to K8s automatically
- Provides a UI dashboard to see app state, rollouts, rollbacks
- Define your app state as YAML in Git → ArgoCD keeps cluster in sync
- Pipeline is not the best method for K8s — use ArgoCD instead
- Supports: production, staging, and development environments
- Part of the Argo Project — ArgoCD is its core component

Interview Answer

"What most people miss is that a traditional Jenkins pipeline pushes changes to Kubernetes — but ArgoCD pulls changes from Git. That pull-based model is more secure and reliable. If someone manually changes something in the cluster, ArgoCD detects the drift and corrects it automatically."

Faculty Notes Explanation:

ARGOCD - to do deployment in K8S Cluster

GitOps continuous delivery tool for Kubernetes

It automates the deployment of applications to Kubernetes clusters,

Pipeline is not the best method for K8s, use ARGOCD

INTRO:

ArgoCD is a declarative continuous delivery tool for Kubernetes. ArgoCD is the

core component of Argo Project.
It helps to automate the deployment and management of applications in a K8s cluster. It uses GitOps methodology to manage the application lifecycle and provides a simple and intuitive UI to monitor the application state, rollout changes, and rollbacks.
With ArgoCD, you can define the desired state of your Kubernetes applications as YAML manifests and version control them in a Git repository.
ArgoCD will continuously monitor the Git repository for changes and automatically apply them to the Kubernetes cluster.

multiple environments such as production, staging, and development.
It is a popular tool among DevOps teams who want to streamline their Kubernetes application deployment process and ensure consistency and reliability in their infrastructure.

SECTION 3 — Install ArgoCD Using Helm

What It Is

Installing ArgoCD on a Kubernetes cluster using Helm charts.

Why It Exists

Helm makes ArgoCD installation a single command instead of applying dozens of YAML files manually.

Think of It Like This

You are adding ArgoCD's app store to Helm, then using it to install ArgoCD into its own dedicated namespace.

Key Points

- ArgoCD runs in its own namespace called `argocd`
- Helm needs the ArgoCD repo added before installing

- After install → expose the server so you can access the UI
- jq tool is needed to extract the server address from JSON output

Commands — Full Installation Sequence

```
# Step 1 — Add ArgoCD Helm repo
helm repo add argo https://argoproj.github.io/argo-helm

# Step 2 — Update repo list
helm repo update

# Step 3 — Create dedicated namespace
kubectl create namespace argocd

# Step 4 — Install ArgoCD via Helm into argocd namespace
helm install argocd argo/argo-cd --namespace argocd

# Step 5 — Verify everything is running
kubectl get all -n argocd

# Step 6 — Expose ArgoCD server via LoadBalancer
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'

# Step 7 — Install jq (JSON parser tool)
yum install jq -y

# Step 8 — Get ArgoCD server address [reconstructed — cut off in image]
//export ARGOCD_SERVER=`kubectl get svc argocd-server -n argocd -o json | jq --raw-output
```

Interview Answer

"In practice, I always install ArgoCD into its own namespace to keep it isolated from application workloads. Exposing it via LoadBalancer gives the team a public URL to access the dashboard. The reason I use Helm for this is that upgrades become a single `helm upgrade` command instead of manually patching multiple resources."

Faculty Notes Explanation:

Setup KOPS first and with the helm we will setup ARGOCD

Setup KOPS

Install HELM

```
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
```

Install ARGOCD using HELM

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
```

```
kubectl create namespace argocd
helm install argocd argo/argo-cd --namespace argocd
kubectl get all -n argocd
```

EXPOSE ARGOCD SERVER:

```
kubectl patch svc argocd-server -n argocd -p '{"spec":
{"type": "LoadBalancer"}}'
```

```
yum install jq -y
```

```
//export ARGOCD_SERVER=`kubectl get svc argocd-server -n
argocd -o json | jq --raw-output
[text cut off at bottom of image - reconstructed from context]
```

Quick Reference

TOOL	ONE LINE PURPOSE
Helm	Package manager – installs apps via charts
ArgoCD	GitOps tool – syncs Git to K8s cluster
Chart	Bundle of K8s manifest files
Release	A deployed instance of a chart
Namespace	Isolated space inside the cluster

ArgoCD — Access, Login & App Setup Notes

SECTION 1 — Expose ArgoCD Server

What It Is

Make the ArgoCD UI accessible from outside the cluster via a public URL.

Why It Exists

By default ArgoCD is internal only. You need a LoadBalancer to access the dashboard from your browser.

Key Points

- Patch the service type to `LoadBalancer` → AWS creates an ELB automatically
- `jq` tool extracts the hostname from JSON output
- The hostname is your ArgoCD browser URL

Commands

```
# Expose ArgoCD server as LoadBalancer
```

```
kubectl patch svc argocd-server -n argocd -p '{"spec":
{"type": "LoadBalancer"}}'

# Install jq (JSON parser)
yum install jq -y

# Get the LoadBalancer URL (commented out alternative method)
//export ARGOCD_SERVER='kubectl get svc argocd-server -n
argocd -o json | jq --raw-output
'.status.loadBalancer.ingress[0].hostname''

//echo $ARGOCD_SERVER

# Actual command to get LoadBalancer URL
kubectl get svc argocd-server -n argocd -o json | jq
--raw-output .status.loadBalancer.ingress[0].hostname

# The above command will provide load balancer URL to access
.GO CD
```

Interview Answer

"In practice, exposing ArgoCD via LoadBalancer on AWS automatically provisions an ELB. I use jq to parse the JSON output and extract just the hostname — otherwise you're digging through pages of JSON output manually."

Faculty Notes Explanation:

EXPOSE ARGOCD SERVER:

```
kubectl patch svc argocd-server -n argocd -p '{"spec":
{"type": "LoadBalancer"}}'
```

```
yum install jq -y
```

```
//export ARGOCD_SERVER='kubectl get svc argocd-server -n
argocd -o json | jq --raw-output
'.status.loadBalancer.ingress[0].hostname''
```

```
//echo $ARGOCD_SERVER
```

```
kubectl get svc argocd-server -n argocd -o json | jq
--raw-output .status.loadBalancer.ingress[0].hostname
```


The above command will provide load balancer URL to access
.GO CD

SECTION 2 — Get ArgoCD Password

What It Is

Retrieve the auto-generated admin password that ArgoCD creates at install time.

Why It Exists

ArgoCD stores its initial password as a Kubernetes Secret in base64 format.
You must decode it to log in.

Key Points

- Password is stored as a Kubernetes Secret named `argocd-initial-admin-secret`
- It is base64 encoded — must decode with `| base64 -d`
- Username is always: `admin`
- Store in a variable with `export` for easy reuse

Commands

```
# Direct command — get and decode password in one line
kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d
```

```
# Store password in a variable
export ARGO_PWD='kubectl -n argocd get secret
argocd-initial-admin-secret -o jsonpath="{.data.password}" |
base64 -d'
```

```
# Print the password
echo $ARGO_PWD
```

```
# The above command to provide password to access argo cd
```

Interview Answer

"What most people miss is that the ArgoCD password is base64 encoded inside a Kubernetes Secret — not plain text. You have to pipe it through `base64 -d` to

decode it. I store it in an environment variable so I don't have to run the command every time."

Faculty Notes Explanation:

TO GET ARGO CD PASSWORD:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o  
jsonpath="{.data.password}" | base64 -d
```

```
export ARGO_PWD='kubectl -n argocd get secret  
argocd-initial-admin-secret -o jsonpath="{.data.password}" |  
base64 -d'
```

```
echo $ARGO_PWD
```

The above command to provide password to access argo cd

SECTION 3 — Login & Create App in ArgoCD UI

What It Is

Log into the ArgoCD dashboard and connect it to a GitHub repo to start automated deployments.

Why It Exists

Once connected to Git, ArgoCD watches for changes and deploys them to the cluster automatically — no manual `kubectl apply` needed.

Key Points

- Open LoadBalancer URL in browser → ArgoCD UI appears
- Login: username = admin, password = from above command
- Create an app by filling in these fields:

FIELD	VALUE

Application Name	bankapp
Project Name	default
Sync Policy	Automatic
Repository	https://github.com/ReyazShaik/ar-deploy
Path	./ (root of the repo)
Cluster URL	(your cluster URL)
Namespace	default

Commands After App Creation

```
# Verify pods were created automatically by ArgoCD
kubectl get po --> it created automatically from argocd
```

```
# Get services - copy paste the elb on browser
kubectl get svc --> copy paste the elb on browser
```

Interview Answer

"The reason ArgoCD's Sync Policy set to Automatic matters is that the moment I push a change to the GitHub repo — say changing replicas from 2 to 5 in `deploy.yml` — ArgoCD detects it within seconds and applies it to the cluster without anyone running a single `kubectl` command."

Faculty Notes Explanation:

Open ArgoCD load balancer

username: admin and password from above command

```
create app --> Application Name --> bankapp --> Project Name
--> default -->
Sync Policy --> Automatic --> Repository -->
https://github.com/ReyazShaik/ar-deploy.git
--> Path --> ./ --> Cluster URL --> Namespace --> default
```

```
kubectl get po --> it created automatically from argocd
```

```
kubectl get svc --> copy paste the elb on browser
```

```
now modify replica as 5 in GitHub deploy.yml , automatically  
argocd will deploy
```

SECTION 4 — History and Rollback

What It Is

View past deployments and roll back to any previous version from the ArgoCD UI.

Why It Exists

If a bad deployment goes live, you need to quickly revert to a working version without manually editing Git files.

Key Points

- Every deployment is recorded in ArgoCD history
- Rollback is done directly from the UI — no commands needed
- Click: History → Three dots → Rollback

Steps

```
ArgoCD UI → Click on App → History and RollBack → Three dots →  
Rollback
```

Interview Answer

"In practice, rollback in ArgoCD is a 3-click operation from the dashboard. The reason this matters in production is speed — if something breaks after a deploy, I can roll back to the last known good state in under 30 seconds without touching the terminal."

Faculty Notes Explanation:

```
History and RollBack
```

```
-----
```

```
Click on history and roll back --> three dots --> rollback
```

Full Workflow Summary

ARGOCD COMPLETE SETUP FLOW

- 1 | Expose server → `kubectl patch svc (LoadBalancer)`
- 2 | Get URL → `kubectl get svc + jq`
- 3 | Get password → `kubectl get secret + base64 -d`
- 4 | Login UI → admin / decoded password
- 5 | Create App → Connect GitHub repo + Automatic sync
- 6 | Verify → `kubectl get po (pods auto-created)`
- 7 | Test GitOps → Change `deploy.yml` in GitHub → auto deploy
- 8 | Rollback → History → Three dots → Rollback